

CHAPTER 1

INTRODUCTION

1.1 Overview

With each new day, opportunities are available to be claimed by qualified applicants. An infinite number of applications and resumes flood the HR professionals with every job opening. This places a burden on the HR personnel and makes the manual process tedious and prone to human bias and errors. This program aims to automate resume analysis and candidate shortlisting using data mining techniques.

The main goal of the project is to build a system that will automate the screening, ranking, and categorization of resumes versus job descriptions. Using data mining techniques on structured and unstructured data, information will be extracted from resumes to compare candidates' qualifications with the job requirements, hence generating an ordered list of applicants in a ranking order. This helps better recruitment processing and adds consistency and objectivity to the evaluation.

1.2 Background

Recruitment for every organization is a vital part of hiring employees to improve productivity and have long-term impacts on the growth of that organization. Traditionally, resumes were screened manually by HR professionals who had to read and assess large volumes of resumes. But with increased job applicants, and the tendency of most organizations to recruit quickly, resume screening has become an inefficient and slow process full of human error and bias. Thus, there is a need for an automated system capable of reducing workload and accurately shortlisting candidates.

Data mining offers a strong method to automate resume screening by identifying valuable patterns within large datasets. Resumes include different kinds of unstructured information, such as educational qualifications, technical skills, work experiences, and personal achievements. Data mining and text-processing techniques help to extract this information, cleanse it, and transform it into structured data which is suitable for analysis. Some of these techniques are keyword extraction, feature selection, classification, and clustering which would help in matching the potential candidate profiles to specific job requirements.

Technological advancements in machine learning and natural language processing (NLP) strengthen these automated systems even further. Such intelligent systems are able to judge the relevance of a given CV, rank applicants, and give recommendations based on objective measures, thus reducing human intervention. More organizations are now turning to these data-driven approaches for streamlining processes, reducing bias, and improving recruitment quality.

This project is aimed at developing a data mining element automated resume screening model. The system extracts important data from resumes and processes them to evaluate the candidate against predefined criteria. The project is aimed at implementing data-mining algorithms to make recruitment a faster, scalable, and more accurate process-in addition to being a reliable tool for supporting HR departments. Overall, this project provides practical proof of data mining as a technology that solves real-world problems in management in human resources.

1.3 Need for a Resume Screening Using Data Mining

- **Rising Volume of Applications**

Most companies receive hundreds or in some cases thousands of applications for a single job posting. To read and screen that amount manually takes a very long time and is very inefficient.

- **Time-Consuming Manual Method**

A lot of time is wasted by HR staff doing manual readings of resumes, thereby lengthening the overall recruitment cycle and increasing hiring costs.

- **High Risk of Human Error**

While manually screening resumes, it is very easy to miss qualified candidates or misinterpret skills due to fatigue or workload.

- **Inconsistent Evaluation**

Recruiters may see different things on a CV and perceive a candidate in contrasting ways, thereby allowing inconsistency in shortlisting decisions having an element of bias.

- **Need to be Strong Against Hiring Biases**

Normal screening can often introduce biases that go unchecked against other forms of discrimination arising from gender, age, background, or formatting of the resume. Thus, assessing through a data-driven approach provides a less subjective approach.

- **Hard to Manage Unstructured Data**

Resumes are mostly unstructured and variable in format. Therefore, HR teams find it really difficult to get this important information efficiently through screening.

- **Required for Speedy Recruitment**

Companies need to get roles filled quickly in order to keep the workunching, which should be in the front of every last entrance.

- **Improving Quality of Hiring**

Data mining helps in matching candidate profiles accurately to job requirements in improving the quality of shortlisted candidates.

- **Scalability for Large Organizations**

Big companies require a system that works with bulk applicants for numerous roles at the same time. This is provided by automated screening.

- **Apt Resource Utilization of HR Resources**

By automating mundane processes, HR teams can invest more in processes like interviewing, talent engagement, and decision-making instead of wasting time reading resumes.

- **Growing Demand for Data-Driven Decision-Making**

Resume screening using data mining with analytics and automation is keeping pace with modern organizational trends of sound decision-making.

- **Cost Efficacy**

Automation tends to greatly reduce the manpower employed in initial screening and therefore with it effectively reduces operational and recruiting costs.

1.4 Purpose of the Project

The intent of this project is to develop an efficient, automated means of resume screening via data mining techniques so they may support the procedure of recruitment. Manually done, the nature of such screening is slow, inconsistent, and allows for many human errors, especially when handling a large number of resumes. For this reason, the project's next main purposes will include the extraction of critical information from resumes and the analysis and perfect matching of candidate profiles with job requirements.

Given the daunting task of candidate evaluation and ex post examination of academic and working experience, data mining would bring about an unbiased, data-driven check of potential skills involved with the sooner-than-expected achievement of the hiring objective, which intends to not only accelerate the hiring workflow and accuracy but also prioritize fairness with possible human bias transcended. This will help to assist in the sorting of the most useful candidates and narrow down the employment process essentially and empirically.

1.5 Significance:

- Reduces the time required for initial resume filtering, thus helping in increasing the efficiency in recruitment.
- Finds higher accuracy in identifying candidates whose skills are more relevant to the job requirements.
- Reduces human bias and provides fair and objective shortlisting.
- Handles many resumes easily, which is important for bulk hiring.
- Extracts insights from unstructured resume data using data mining techniques.
- Presents data-driven and consistent evaluation to help HR decisions.
- Improves the overall quality of hiring by giving priority to the best-fitting candidates.

1.6 Scope of the Project

- Collating and pre-processing resume datasets
- Identifying Key Attributes such as Skills, Experience, Education, and Keywords
- Analyzing resumes through Data Mining and Text Mining Techniques
- Applying classification, ranking, or matching Algorithms to Analyzed Candidates.
- Evaluation of system performance by accuracy, precision, recall, or similarity scores.
- Development of an automated, scalable system to optimize the efficiency of the recruitment process.
- Shortlists and insights for aiding HR teams in decision-making.

1.7 Problem Statement:

To develop a Resume Screening using the techniques like Data Mining and Machine Learning, this project aims to automatically analyze and classify resumes based on job requirements.

1.8 Objectives

1. To Reduce Time and Effort

Automate the manual process of screening resumes to speed up candidate selection.

2. To Improve Accuracy and Consistency

Use NLP and machine learning to extract relevant skills and qualifications, ensuring consistent evaluations across all resumes.

3. To Minimize Human Bias

Reduce subjectivity and unconscious bias in hiring decisions through data-driven candidate ranking.

4. To Ensure Data Privacy and Security

Address ethical concerns by implementing safeguards for personal and sensitive candidate information.

CHAPTER 2

LITERATURE REVIEW

2.1 RESUME SCREENING USING NLP AND SIMILARITY SCORING.

Articles Based on This Techniques:

- "Data Driven Resume Analyzer for Student Career Optimization"- Ajay Shanker, Mukesh Kumar Sinha, Mayank Bhusan.
- "HR Tech Analyst: Automated Resume Parsing and Ranking System through NLP"- Akhil Goyal, Abhishek Seth, Arpit Jain, Deepak Kumar, Avneesh Gautam.

Method:

A Resume Analyzer System is developed by integrating machine learning techniques within a web-based platform to improve the quality of resumes for students and job seekers by aligning them with job roles and industry expectations [1]. The system adopts a data-driven approach that enables automated resume evaluation and personalized feedback based on current hiring practices [1].

The system is implemented using Natural Language Processing (NLP), Machine Learning (ML), MongoDB, and ReactJS technologies. Users securely register and log in before uploading resumes for analysis. MongoDB is used as the backend database to store resume content, user profiles, and domain-specific skill information, as it efficiently handles diverse and semi-structured resume data [1].

After upload, resumes are processed by the Resume Analysis Module, which applies NLP techniques such as text parsing and Named Entity Recognition (NER) to extract key information including education, experience, certifications, and skills [2]. These extracted features are essential for structuring unorganized resume data into meaningful components for analysis.

The extracted data is further analysed using semantic similarity techniques such as TF-IDF, cosine similarity, and word embedding models like Word2Vec to compare resume content with job descriptions and identify skill gaps [2]. Keyword relevance, resume structure, and language quality are also evaluated to determine overall resume effectiveness.

A Recommendation Engine generates customized feedback by identifying missing or underrepresented skills and poorly structured sections. Unlike traditional keyword-based systems, the recommendations consider contextual meaning and job-specific requirements, leading to more relevant and actionable suggestions [1]. The Resume Ranking Module assigns scores based on content relevance, keyword density, grammar, and layout effectiveness.

The frontend of the system is developed using ReactJS to provide a responsive and user-friendly interface for resume upload and feedback visualization. Secure authentication and encryption mechanisms protect user data, while MongoDB ensures scalability and fast access. The system continuously learns from newly uploaded resumes and job descriptions, enabling it to adapt to evolving recruitment trends and maintain high accuracy [1].

Benefits:

- **High Accuracy and Smart Feedback:** Taps up to 98% accuracy through this contextual NLP and ML for job-specific improvement suggestions.
- **Real Time, Personalized Recommendations:** Provide personalized recommendations that can be used for all their uploaded resumes and the specific job that they are targeting.
- **Secure and Scalable Design:** Built with MongoDB and ReactJS, so fast performance, safety of user data guarantees different flexible data types for use.
- **Dynamic Learning:** Continually updating its models and skill-sets as new hiring trends and requirements change.

Disadvantages:

- **Quality Dependent:** There may be an accuracy drop when resumes are poorly formatted or inconsistent with the information provided in them.
- **Soft Skill Limitations:** It would not help the measurement of non-countable attributes like leadership or even communication skills.
- **High Resource Usage:** NLP and ML computations consume a lot more power, especially with larger user bases.
- **Industry-Specific Conundrums:** Might not be able to grasp niche or hyper-specialized job descriptions fully without manual tuning.

2.2 Natural Language Processing and Support Vector Machine

Papers Using the Techniques:

- “Resume Screening for Automatic Software Engineering Position Using Natural Language Processing, Word Matching, Character Positioning, and Regex” – Pant, Dipendra, Dhiraj Pokhrel, and Prakash Poudyal
- “Automating Resume Classification Using Machine Learning Algorithm” – Akram, Nida

Methodology:

An integrated approach involving machine learning, natural language processing (NLP), and statistical similarity techniques is proposed for automatic resume screening. Automated resume screening systems are widely used to reduce manual effort and bias by systematically collecting, storing, and analyzing resumes uploaded through a web-based portal into a centralized database for large-scale management [2].

After data collection, the resumes undergo a data preprocessing phase. Preprocessing includes tokenization, stopword removal, stemming, and lemmatization to clean and normalize the raw textual data. Tokenization using NLP libraries such as NLTK breaks the resume text into meaningful units, enabling effective understanding of resume structure and content. These preprocessing steps are essential for improving semantic consistency and preparing textual data for further analysis [2], [3].

Following preprocessing, the resumes are processed by the candidate recommendation engine, which forms the core of the system. TF-IDF (Term Frequency–Inverse Document Frequency) is applied to convert resume text into numerical feature vectors by highlighting important and job-relevant terms. TF-IDF is a commonly used technique in resume screening and information retrieval for identifying discriminative keywords within large document collections [2], [3].

Cosine similarity is then employed to compute the similarity between vectorized resumes and job descriptions. This technique measures the cosine of the angle between two vectors in a multidimensional space to quantify how closely a candidate’s resume matches the employer’s job requirements. A higher cosine similarity score indicates a stronger match, enabling effective ranking of resumes according to relevance [2].

To further enhance system performance and classification reliability, a supervised learning algorithm—Random Forest Classifier—is incorporated. Random Forest is an ensemble learning method that constructs multiple decision trees during training and aggregates their predictions to improve accuracy and reduce overfitting. This approach enables the model to learn patterns from historical resume data and improve resume classification and ranking effectiveness [3].

The complete workflow includes user authentication, resume upload, preprocessing, feature extraction, similarity computation, classification, and ranking. Employers can submit job-specific queries, and the system automatically returns a ranked list of resumes along with visual summaries such as percentage match charts. This intelligent automation significantly reduces the time and effort traditionally associated with manual resume screening while improving consistency and accuracy [2], [3].

Advantages:

- Speeds up recruitment: automation of the painstaking task of manual screening of resumes now enhances hiring speed.
- Enhanced accuracy: assisted by TF-IDF and cosine metric ensure in that phase that only the most relevant resumes get shortlisted.
- Scalable and adaptable: Can accommodate thousands of resumes and adapt to multiple job descriptions using a trained model.
- Improved Decision Making: Random Forest Classifier further improves predictive performance, lowering the threshold for unqualified misses.

Disadvantages:

- Lacks proper context comprehension: Cosine similarity and TF-IDF are subject to scores occurring from keyword presence, which may nullify context or nuance in qualification.
- Poor data preprocessing: The system requires high-quality clean data; resumes with faulty formatting reduce usefulness.
- Potential for Bias: Machine learning models trained on flawed data can reproduce and even amplify existing biases in candidate selection.
- Not empirically validated in the real world: The system is based on textual content of resumes, where no real mechanism exists to validate skills or experience in practice.

2.3 Resume Net: A Learning Based Framework

Related Papers:

- "Resume Net: A Learning-based Framework for Automatic Resume Quality Assessment"-Yong Luo, Huaizheng Zhang
- "Natural Language Processing and Text Mining to Identify Knowledge Profiles for Software Engineering Positions"-Rogelio Valdez-Almada, Oscar M. Rodriguez-Elias

Methodology:

The learning-based resume evaluation framework described above is primarily derived from ResumeNet, proposed by Luo et al. [4]. ResumeNet introduces a deep neural architecture for automatic resume quality assessment under severe label scarcity. The framework processes a large corpus of resumes with only a small fraction labeled by experts, addressing class imbalance using semi-supervised learning techniques. The model incorporates word- and sentence-level embeddings (Universal Sentence Encoder), attention mechanisms to model resume components such as skills and work experience, and a consistency sub-network that verifies claimed skills against job descriptions using cosine similarity. Structured features such as education level, years of experience, and awards are fused with learned representations through fully connected layers to predict a resume quality score. To mitigate the lack of labeled data, ResumeNet further employs contrastive learning, triplet loss, and manifold regularization, improving representation robustness and generalization. Performance evaluation is conducted using ROC, F1-score, and Average Precision metrics [4].

The technical knowledge extraction and profiling component aligns with the work of Valdez-Almada et al. [5], which presents a rule-based NLP and text-mining framework for generating knowledge profiles from resumes for software engineering roles. Their approach follows a conventional text-mining pipeline involving preprocessing, knowledge extraction, categorization, and profile generation. Using Stanford CoreNLP and a custom knowledge base, the system identifies predefined technical knowledge such as programming languages, databases, IDEs, and testing frameworks, including handling multiple lexical variants of the same concept (e.g., *PostgreSQL* and *Postgres*) through regular expressions. The extracted technical knowledge is used to build a dynamic and modifiable knowledge profile, enabling recruiters to efficiently shortlist candidates based on technical fit. The study also reports significant efficiency improvements, processing resumes nearly four times faster than manual screening while maintaining comparable accuracy [5].

Together, these two approaches complement each other: ResumeNet focuses on learning-based resume quality assessment under limited supervision, while the KP Generator emphasizes rule-based technical knowledge extraction and profiling, making the combined methodology scalable, flexible, and practical for real-world recruitment systems [4][5].

Benefits

1. Automating Resume Analysis: Both models eliminate or reduce the manual screening, thus saving time and effort for HR teams.
2. Incorporating Deep Learning and NLP: While KP Generator uses Stanford CoreNLP and rule-based knowledge extraction in its operation, ResumeNet employs deep neural architectures integrated with attention mechanisms.
3. Scalability: The frameworks can handle massive volumes of resumes with efficient processing, thus making them appropriate in the real world with thousands of applicants.
4. Semi-supervised and rule-based Flexibility: ResumeNet manages the scarcity of labels by manifold regularization and pair/triplet loss training, whereas KP Generator allows different rules-based adjustments of domain-specific knowledge profiles.

Drawbacks

1. Less Labeled Data: A few labeled samples (only 122 out of 10,000) are a downfall for ResumeNet that can undermine model reliability, along with semi-supervised techniques.
2. Reliance on Predefined Rules: Knowledge extraction through rules of KP Generator demands manual framing and updating of the rules, which may not generalize well on multiple/all kinds of unconventional because of diversity in resumes.
3. Contextual Understanding: These systems depend on embeddings or token matching, possibly not working with resumes written in very nonstandard or creative formats that lack specific technical keywords.

2.4 The Automated Parsing of Resume and Ranking System

Author Papers Utilizing This Technique:

- “Web Application for Screening Resume”- Sujit Amin, Nikita Jayakar, Sonia Sunny, Pheba Babu, M. Kiruthika
- “HR Tech Analyst: Automated Resume Parsing and Ranking System through Natural Language Processing”- Nisha B, Manobharathi V, Jeyarajanandhini B, Dr. G. Sivakamasundari

Method:

1. Resume Collection and Input Handling

The resume screening system begins with the collection of resumes uploaded by job applicants in PDF or Word formats. Since these formats are not directly machine-readable, documents are converted into plain text using tools such as pdf2txt, PyPDF2, or python-docx. For web-based storage, base64 encoding is used to store resumes in relational databases. This ensures secure storage and enables subsequent NLP-based processing of unstructured resume data [6], [7].

2. Pre-processing of Text

Once text extraction is completed, preprocessing is applied to improve data quality and consistency. This includes converting text to lowercase, tokenization, removal of stop words, cleaning special characters and extra spaces, and normalizing formats such as dates. These preprocessing steps reduce noise and prepare the text for accurate information extraction and ranking [6], [7].

3. Information Extraction using NLP

The system employs NLP techniques to extract structured information from resumes. Section-based segmentation is used to divide resumes into logical sections such as education, skills, and work experience. Named Entity Recognition (NER) identifies entities including university names, company names, job roles, and skills. Contextual and semantic analysis links extracted skills to corresponding job experiences. Additionally, applicants are allowed to manually verify and edit extracted information to improve accuracy before final submission [6], [7].

4. Morphological Analysis

Morphological processing is applied to normalize textual variations. Stemming reduces words to their root forms (e.g., running → run), while lemmatization considers grammatical context to derive meaningful base forms (e.g., mice → mouse). This improves keyword matching, entity recognition, and consistency across resumes [7].

5. Training and Custom Model Development

Annotated resumes are used to train domain-specific NLP models. Tools such as Dataturks are employed to label entities and convert resumes into JSON format. These annotated datasets are then used to train customized SpaCy models tailored for resume parsing tasks. Semi-supervised learning is applied to reduce manual labeling effort while maintaining extraction accuracy, especially in large-scale recruitment systems [6], [7].

6. Job Description Parsing

Job descriptions provided by recruiters undergo the same NLP pipeline as resumes. Relevant entities such as required skills, education, and experience are extracted and stored in structured formats. This uniform processing enables effective and fair matching between candidate resumes and job requirements [6], [7].

7. Scoring and Ranking

Candidate resumes are evaluated against job descriptions using multiple criteria, including skill match, education level, years of experience, certifications, and project relevance. Weighted scoring formulas are applied to quantify candidate suitability. Based on these scores, resumes are ranked from highest to lowest relevance, allowing recruiters to identify top candidates efficiently [6], [7].

8. Ranking and Visualization

The ranked candidate list is presented exclusively to recruiters. Visualization techniques such as tables, charts, and graphs are used to summarize applicant distributions and ranking outcomes, enabling quicker interpretation and data-driven decision-making [7].

9. Recruiter Interface and Automation

A dedicated recruiter interface allows viewing ranked candidates, applying custom filters, and managing shortlisted applicants. Automation features streamline the recruitment workflow by reducing manual effort and accelerating candidate selection, thereby improving hiring efficiency and consistency [6], [7].

Advantages

1. Efficiency and Time-Saving: Automated resume screening hence heavy manual work is cut down.
2. Accurate from NLP Perspective: Custom-trained NLP models improve the accuracy of data extraction and classification immensely.
3. Customizable and Scalable: Parsing rules and ranking criteria can be set to various industries and job roles.
4. Enhanced Candidate Experience: Higher candidate experience as faster processing and response time improves the overall face of the hiring process.

Disadvantages

1. Complex Initial Setup: Training the NLP model and developing tailored rules need heavy initial effort and technical skill.
2. Data Privacy and Security Risks: Working with sensitive resume data necessitates strict encryption and compliance measures.
3. Limited Understanding of Creative Layouts: Resumes that are extravagantly stylized in their layout may yield incorrect parsing results.
4. Potential Bias in Training Data: Heavily impacts performance and fairness due to quality and diversity training dataset.

CHAPTER 3

REQUIREMENTS

3.1 Functional Requirements

1. User Management

- The system shall allow HR/recruiters to create and log in to their accounts.
- The system shall authenticate users using email/username and password.

2.Resume Upload & Input Handling

- The system shall allow users to upload resumes in formats like PDF, DOC, DOCX, TXT.
- The system shall extract text from uploaded resumes using NLP/text-processing tools.

3.Data Preprocessing

- The system shall clean and preprocess extracted text (remove stopwords, punctuations, etc.).
- The system shall identify key sections like skills, experience, education, and certifications.

4.Job Description Input

- The system shall allow users to upload or type a job description (JD).
- The system shall pre-process and extract keywords from the JD.

5.Resume Screening & Matching

- The system shall compare each resume with the JD using data mining/ML algorithms (TF-IDF, cosine similarity, classification, clustering, etc.).
- The system shall score and rank resumes based on relevance.
- The system shall identify and highlight missing or required skills.

6.Results & Reporting

- The system shall display a list of shortlisted resumes.
- The system shall generate a similarity score for each candidate.
- The system shall provide detailed analytics (skills match %, experience match, keyword frequency, etc.).
- The system shall allow the user to download or export results (PDF/CSV).

7. Notifications

- The system shall notify the recruiter when screening is completed.
- The system shall notify users when resumes fail to upload or have parsing errors.

8. Databases & Storage

- The system shall maintain user data, resume data, and resume screening result in the database.
- The system shall allow users to view the screened resumes.

3.2 Non-Functional Requirements**1. Performance**

- The system shall process and screen resumes within 5 seconds per resume (or a configurable limit).
- The system shall support batch screening of at least 200 resumes at once.

2. Reliability & Availability

- The system shall have 99% uptime for cloud-based deployment.
- The system shall automatically handle parsing errors and resume processing.

3. Security

- The system shall encrypt stored resumes and user data.
- The system shall adhere closely to secure authentication protocols (hashed passwords, plain text not to be used).
- Access to candidate information shall be exclusive to authorized users.

4. Usability

- The system's interface will be simple for HR personnel who are less technical and convenient to use.
- Screen navigation and actions shall require no more than 3 clicks.

5. Scalability

- The system shall scale on demand to handle increasing resume numbers without degradation in performance.
- The system shall integrate with external systems and ATS platforms.

6. Maintainability

- The system code shall be designed modularity-wise for easy updates.
- The system should make it possible to update ML models without complex changes in the system.

7. Accuracy

- This system for resume-JD match must achieve 80–90% accuracy in Levatin keyword detection.
- The system should have as minimal false positives and false negatives for candidate ranking as possible.

8. Compatibility

- The system shall support resumes of all major formats (PDF, DOCX, etc.).
- The system shall interface well with browsers like Chrome, Firefox, and Edge.

CHAPTER 4

PROJECT DESIGN

Design of a project is the most critical state during the life cycle of software development. It works as a bridge between understanding the concept and putting it to real implementation. The design of the Resume Screening System using Data Mining entails the components of accuracy, efficiency, and scalability that are essential to the automation of evaluation and ranking of resumes against job criteria. This chapter provides discussions about the system architecture and data flow; it also mentions the design elements that ensure seamless interaction between data collection, preprocessing, feature extraction, and classification modules.

The proposed system observes an architecture based on layers to maintain clear separation of concerns. Thus, it consists of the presentation layer (user interface to recruiters), the processing layer (data mining and machine learning for resume analysis and matching), and the data layer (containing resume repositories, job descriptions, and extracted features in a database). Each layer functions independently while providing secure and efficient communication needed for smooth resume screening and decision support. The following sections explain every design aspect with the support of conceptual diagrams and analytical explanations.

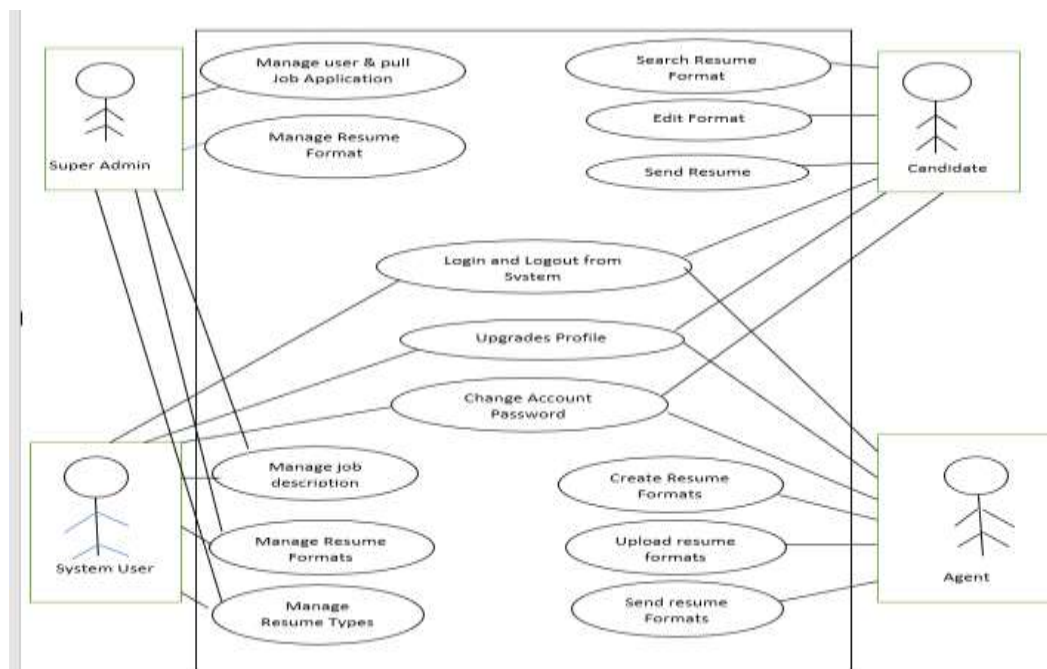


Fig 4.1 Use Case Diagram

1. Super Admin

It is the highest in command. This Super Admin has the following abilities:

- Manage all users in the system
- Observe and retrieve job applications
- Manage and modify the formats of resumes

This role is meant to ensure that the platform is in working condition and the users obey the rules.

2. System User

Generally, the system users are either HR or internal staff. They:

- create and control job descriptions.
- organize resume types and formats.

All the system does is prepare them to be used by candidates and agents in-house with the standardized resume templates.

3. Candidate

It is a candidate that uses the system to develop the resume. Candidates can do the following:

- Search among the formats of resumes
- Edit and update the resume
- Send it to job applications

This gives candidates the opportunity to write resumes in the format needed by the job.

4. Agent

Agents are those who create resumes for others. They can:

- Create new resume formats
- Upload Resumes templates
- Send formatted Resumes.

This role enables candidates to by-pass calls for ready-made or customized formats of resumes.

Basic Functions for all Users

All users share these common basic functionalities of the system:

- Login and Logout

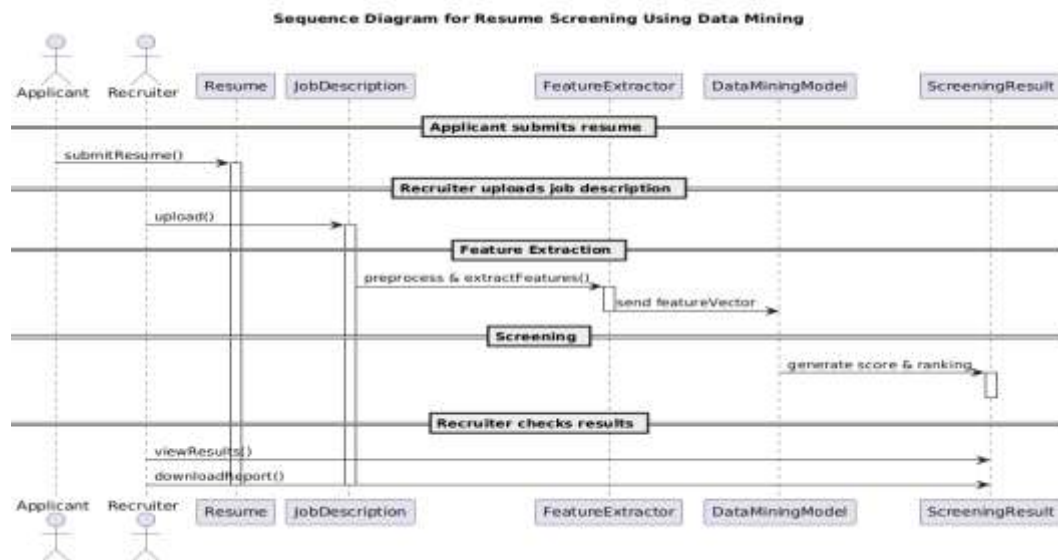


Fig 4.2 Sequence Diagram

This sequence diagram depicts how resumes are screened in a stepwise fashion using a data-mining model.

1. Applicant submission of resume

- The Applicant uploads the resume.
- The Resume object receives the submitResume() request.

2. Job description uploaded by recruiter

- Job description provided by the Recruiter.
- The Job Description object receives the upload() request.

3. Feature extraction

- The Feature Extractor takes both resume and job description.
- It makes use of preprocess & extractFeatures().
- Feature vector (structured data) is sent to the Data Mining Model.

4. Screening

- The Data Mining Model processes the feature vector.
- Then it generates a score, match percentage and ranking.
- Finally sends it to Screening Result.

5. Recruiter views results

- Results are to be viewed by the Recruiter using viewResults().
- The Recruiter can also download a screening report using downloadReport().

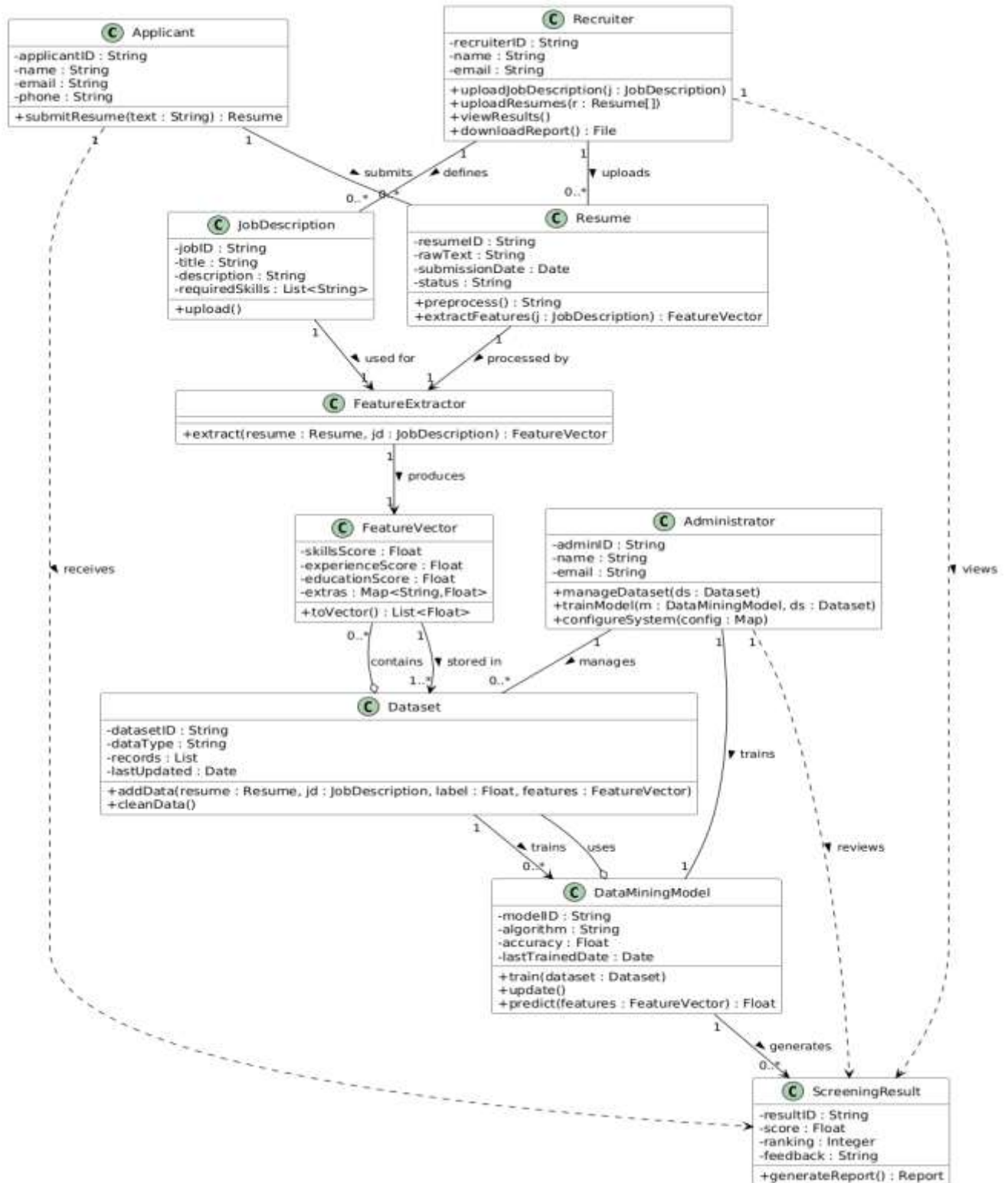


Fig 4.3 Class Diagram

This is a UML Class Diagram for a Resume Screening System using Data Mining. It depicts the main classes with their attributes, methods, and relationships which explain how resumes are processed and evaluated.

1. Applicant

Purpose: Represents a job seeker.

Attributes:

- applicantID, name, email, phone

Method:

- submitResume(text) : Resume` → submits resume text to the system

Relationships:

- An applicant **submits multiple resumes**
- An applicant **receives screening results**

2. Recruiter

Purpose: Represents the employer or HR personnel.

Attributes:

- recruiterID, name, email

Methods:

- uploadJobDescription(j)
- uploadResumes(r[])
- viewResults()
- downloadReport()

Relationships:

- Defines **Job Descriptions**
- Uploads **Resumes**
- Views **Screening Results**

3. JobDescription

Purpose: Stores job requirements.

Attributes:

- jobID, title, description, requiredSkills

Method:

- upload()

Relationships:

- Used along with resumes for **feature extraction**

4. Resume

Purpose: Stores applicant resume details.

Attributes:

- resumeID, rawText, submissionDate, status

Methods:

- preprocess() → cleans resume text
- extractFeatures(jd) → extracts features based on job description

Relationships:

- Processed by **FeatureExtractor**

5. Feature Extractor

Purpose: Extracts relevant information from resumes.

Method:

- extract(resume, jd) : FeatureVector

Relationships:

- Uses **Resume** and **JobDescription**
- Produces **FeatureVector**

6. FeatureVector

Purpose: Holds numerical values used for prediction.

Attributes:

- skillsScore
- experienceScore
- educationScore
- extras

Method:

- toVector() → converts data into numeric vector

Relationships:

- Stored in **Dataset**

7. Dataset

Purpose: Stores training and prediction data.

Attributes:

- datasetID, dataType, records, lastUpdated

Methods:

- addData(resume, jd, label, features)
- cleanData()

Relationships:

- Contains multiple FeatureVectors
- Used to train DataMiningModel
- Managed by Administrator

8. Administrator

Purpose: Manages system and model training.

Attributes:

- adminID, name, email

Methods:

- manageDataset(ds)
- trainModel(m, ds)
- configureSystem(config)

Relationships:

- Manages **Dataset**
- Trains **DataMiningModel**
- Reviews **ScreeningResults**

9. DataMiningModel

Purpose: Applies machine learning or data mining algorithms.

Attributes:

- modelID, algorithm, accuracy, lastTrainedDate

Methods:

- train(dataset)
- update()
- predict(features) : Float

Relationships:

- Uses **Dataset**
- Generates **ScreeningResult**

10. ScreeningResult

Purpose: Final output of resume screening.

Attributes:

- resultID, score, ranking, feedback

Method:

- generateReport()

Relationships:

- Viewed by **Recruiter**
- Received by **Applicant**

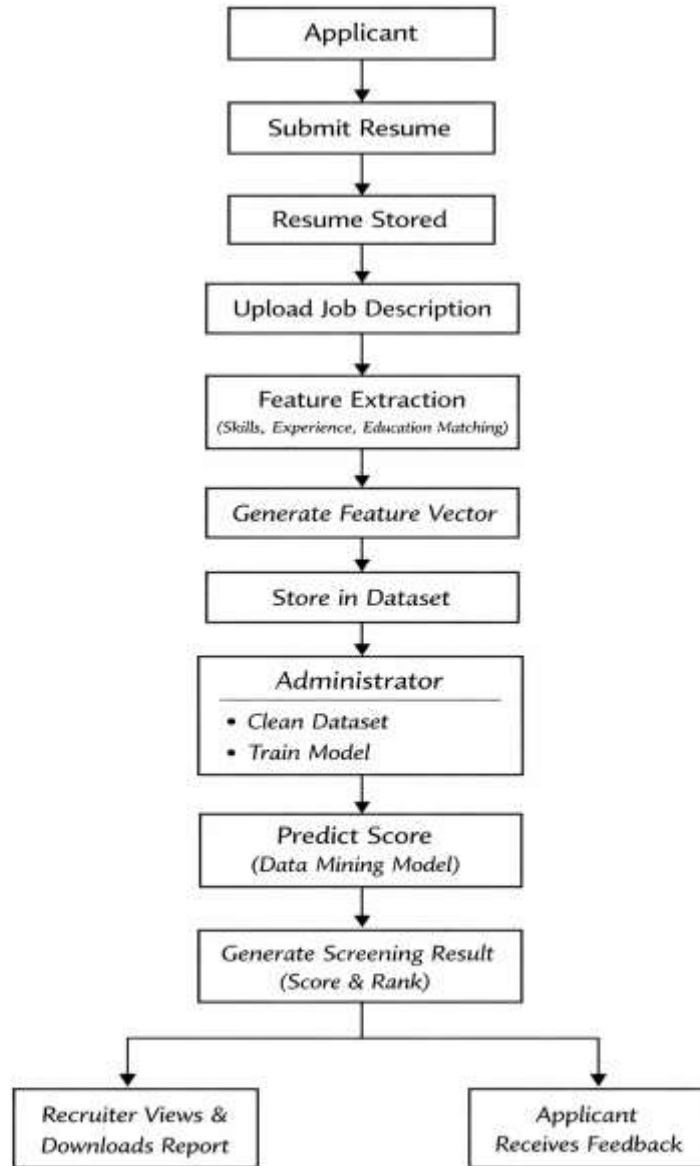


Fig 4.4 Flow Diagram

This is the **Flow diagram of a Resume Screening System using Data Mining**. This flow diagram represents stepwise movement of information from resume submission to final results.

Flow in step:

1. Applicant

The process begins part of the applicant.

2. Resume Submission

The applicant submits his/her resume (PDF/Word format) to the system.

3. Resume Saved

The uploaded resume is saved in the system. In this phase, text extraction and basic preprocessing such as cleaning and formatting are done.

4. Upload Job Description

The recruiter uploads the job description which has job requirement details, such as skill-set, experience, and qualification.

5. Feature Extraction

With the help of feature extraction, it further makes important comparisons of the resume to the job description and extracts features like:

- Match of skills
- Match of experience
- Relevance of education

6. Create Feature Vector

These feature extraction components are then deleted into numbers (the feature vector) understandable by the data mining application.

7. Hold in Dataset

Such feature vectors will be stored within a dataset being utilized for training and prediction purposes.

8. Administrator (Skillful Dataset Clean up & Train Model)

The administration body maintains the dataset by cleaning it and gets the mining model into training using past data.

9. Predict Score(Data Mining Model)

Then, using the feature vector, the trained model determines the candidate's achievement.

10. Generate Screening Result (Score & Rank)

Based on the predicted score, ranking is done, and results of the screening are generated.

11. Final Output (Two ways)

- **Recruiter Views & Downloads Report** the recruiter can view shortlisted candidates and download the screening report or
- **Applicant Gets the Feedback** the applicant is updated with the screening status.

Summary:

This flow mainly shows how resumes are processed automatically using data mining techniques, which not only reduce manual work but also improve the efficiency of recruitment.

CHAPTER 5

PROJECT IMPLEMENTATION

Implementing the Resume Screening System involves building the architecture of the whole system, including database design, user authentication, document processing, natural language processing, similarity scoring, and visualization. Python is being used as the main environment for coding as it supports diverse machine learning and text-processing libraries. Streamlit was employed for constructing the web framework to develop an interactive user-friendly interface for human resource professionals and candidates. SQLite manages the backend of the system, which relates to the storage of user credentials, job postings, resume details, screening results, and system logs. It automatically initializes and upgrades on startup, ensuring that the structure of the database includes all the tables that need to exist.

Using PyPDF2 for PDF files and python-docx for Word documents, all resume processing retrieves clean-text with minimal formatting. The next step is extraction, at which point both the job description and resume are subjected to the SBERT model ("all-MiniLM-L6-v2") for creating their semantic embeddings. To assess with cosine similarity how close matches exist between resumes and the job requirements, the matching procedure uses embeddings. For systems where SBERT cannot load, there is an alternative fallback keyword-based similarity method. To assess candidates better, skill extraction and years of estimate experience get combined with the creation of a weighted score based on semantic similarity and important candidate skills. The user defines above what threshold will mark a candidate eligible for the shortlist for the HR user. Visualization of the results was done through Altair charts, ranking graphs, and pie charts which summarize an account of candidate distribution.

The application also offers export options bleeding in CSV, Excel, and PDF reports where pandas, openpyxl, and reportlab are used, respectively. Additional modules comprise role-based login; job-post management; candidate upload option; agent interface; and system-wide notifications. Overall implementation projects a complete workflow from resume upload to final shortlisting as specified for automating and increasing recruitment efficiency.

- Python + Streamlit used for creating interactive web platforms.
- SQLite takes care of writing away to store all records.
- Role-based login for Super Admin, HR, Agent, and Candidate.
- Text extraction from PDF and DOCX using PyPDF2 and python-docx.

SBERT semantic similarity for accurate matching between resumes and job descriptions.

- Skill extraction + experience estimation via regex and pattern matching.
- Weighted scoring model combines similarity + skill importance.
- Altair visualizations for ranking bars, pie charts, and candidate summaries.
- Resume export options (CSV, Excel, PDF) for reporting and documentation.
- Complete workflow including job creation, resume upload, screening, and shortlisting.

5.1 CODE IMPLEMENTATION

5.1.1 Database Initialization Code

```
CREATE TABLE IF NOT EXISTS users(  
id INTEGER PRIMARY KEY AUTOINCREMENT,  
username TEXT UNIQUE,  
password TEXT,  
role TEXT  
)
```

This SQL command creates a table called “users” only if it does not already exist in the database. The table stores user account details with “id” as a unique auto-incrementing primary key, “username” as a unique text field, and “password” and “role” for storing login credentials and user type. This structure helps manage authentication and authorization for the system.

```
CREATE TABLE IF NOT EXISTS job_posts(  
id INTEGER PRIMARY KEY AUTOINCREMENT,  
title TEXT,  
description TEXT,  
skills_required TEXT  
)
```

An SQL statement is executed to create the “job_posts” table for the “Resume Screening System” if it does not exist. It stores job-related information with respect to the job “title”, detailed “description”, and “skills_required”, which are used for reference data for the matching and screening of candidate resumes.

```
CREATE TABLE IF NOT EXISTS screening_results(  
id INTEGER PRIMARY KEY AUTOINCREMENT,  
job_id INTEGER,  
candidate_name TEXT,  
similarity REAL,  
skill_score REAL,  
final_score REAL  
)
```

This SQL query creates the table “screening_results” for Resume Screening System if it is not yet created. It stores the evaluation results of candidates for any given job, where “job_id” links the result to a job post, and similarity, “skill_score”, and “final_score” are the computed matching scores used to rank resumes.

5.1.2 Resume Text Extraction Code

```
def extract_pdf_text(path):  
text = ""  
with open(path, "rb") as f:  
reader = PyPDF2.PdfReader(f).  
for page in reader.pages:  
text += page.extract_text() + "  
return text
```

Reuse of data from a resume file in PDF format is based on using this function for the Resume Screening System. It opens this resume PDF in binary mode, reads each page using PDF reader, and concatenates that into a single string. Finally, this string is returned for doing pre-processing and analysis in continuation with screening the resume.

```
def extract_docx_text(path):  
doc = Document(path).  
return "".join([p.text for p in doc.paragraphs]).
```


This function will open a resume that is uploaded in “.docx” format and read the text contents of the document. It will extract all paragraph texts, then combine the text into a single string in order that it would become useful in use in “resume screening project” for skills, experience, and keywords extraction using data mining techniques.

5.1.3 SBERT Similarity Computation

```
model = SentenceTransformer("all-MiniLM-L6-v2").
def compute_similarity(job_desc, resume_text):
    job_emb = model.encode(job_desc).
    res_emb = model.encode(resume_text).
    score = util.cos_sim(job_emb, res_emb).item().
    return round(score, 4).
```

The provided implementation utilizes a “SentenceTransformer model” for transforming a job description-resume text into numerical embeddings. These embeddings undergo a cosine similarity measure for assessing a match between a resume and job requirement, therefore, facilitating accurate ranking of candidates in the “resume screening” project.

5.1.4 Skill Extraction and Weighted Score

```
def extract_skills(text, skill_list):
    found = [skill for skill in skill_list if skill.lower() in text.lower()].
    return found.
def calculate_final_score(similarity, skill_match_count, total_skills):
    skill_score = skill_match_count / total_skills.
    final_score = (0.7 * similarity) + (0.3 * skill_score).
    return round(final_score, 4).
```

The “extract_skills” function identifies relevant skills from a resume by looking for matches with a predefined skill list. The “calculate_final_score” function takes into account both the “semantic

similarit” of the resume and the job description and the “skill match ratio” to compute a final score, thus allowing for precise ranking of candidates in the “resume screening” project.

5.1.5 Streamlit UI (Screening Section)

```
st.title("Resume Screening System").
job_desc = st.text_area("Job Description").
uploaded_files = st.file_uploader("Upload Resumes", accept_multiple_files=True).
if st.button("Start Screening"):
    for file in uploaded_files:
        if file.type == "application/pdf":
            text = extract_pdf_text(file).
        else:
            text = extract_docx_text(file).
        similarity = compute_similarity(job_desc, text).
        skills = extract_skills(text, ["Python", "ML", "SQL", "Java"]).
        final = calculate_final_score(similarity, len(skills), 4).
        st.write(f"***{file.name}** → Score: {final}").
```

In this code, Streamlit facilitates the design of an interface for a “Resume Screening System” that allows recruiters to input the job description and upload multiple resumes. Text is extracted from each resume and compared with the job description for similarity; matched skill sets are and are considered for final scoring, hence providing an automated ranking of candidates using “data mining techniques”.

5.2 Data set description.

5.2.1 Overview of Dataset:

Resume Screening System using Data mining is a labeled resume data set containing real world text data resumes categorized according to job roles. This data set is saved in a CSV file format. It is used to train, test and evaluate the data mining model in use in the system.

Each record in the dataset represents a single resume, along with its corresponding job category, making it suitable for supervised learning techniques.

5.2.2 Structure of Dataset:

The dataset consists of the following main attributes:

1. Resume

- Contains raw text contents of the candidate's resume;
- Citing information about skills, education, work experience, certifications and projects;
- The attribute serves as the main entry during text preprocessing and feature extraction.

2. Category (Label)

- Represents the job role/domain to which the resume belongs (e.g., Data Scientist, HR, Software Developer, Network Engineer, etc.);
- It acts as the class label for training the data mining model.

5.2.3 Role of Dataset in Project Implementation

1. Data Preprocessing

Before model training, the dataset will undergo the following preprocessing steps:

- Removal of special characters, stop words, and punctuation
- Conversion to lower case
- Tokenizing and normalizing
- Clearing irrelevant or noisy data

This guarantees the clean resume text, ready for features extraction.

2. Feature Extraction

From preprocessed resumes, meaningful features are extracted using techniques such as:

- Term Frequency-Inverse Document Frequency (TF-IDF)
- Keyword and matching skills
- Experience and education term identification

Then, these features are converted into numerical feature vectors representing each resume in machine-understandable form.

3. Dataset Storage and Management

Store extracted feature vectors along with their labels in Dataset module.

The data set gets clean and updated regularly by the administrator to keep improving the model accuracy and reliability.

4. Model Training

The labeled dataset is used to train Data Mining Model (like Naïve bayes, Support Vector Machine, or any other classification algorithm).

The model learns the pattern between resume content and Job category.

5. Prediction and Screening

Following are the steps involved in implementation:

- Once the applicants upload their fresh resumes, it is processed in the same way as training data.
- The trained model then predicts a matching score or category for each resume.
- Based on this prediction, the resumes are ranked, and screening results are generated.

Advantages of Using This Dataset

- Real-world resume text; this increases its relevance in practice
- Labeled categories point towards supervised learning
- Accurate classification of resumes and ranking of resumes
- Flexible in adding new resumes and job roles without alteration

CHAPTER 6

TESTING

During the testing phase of the RSC Application, the validation of each module's functionality, robustness, and reliability was carried out independently. Each of the test cases was carried out with actual resumes, multiple user roles, and real job descriptions. The representations of the description of each test case are given in paragraphs, but if necessary, some important considerations if needed are included in the code.

6.1 Unit Testing and Test Cases

Unit testing was performed to validate individual functions of the Resume Screening System in isolation. Core modules such as resume text extraction, semantic similarity calculation, skill extraction, weighted score computation, and email extraction were tested using controlled inputs to ensure correctness.

Sample Test Case 1 – Resume Text Extraction (PDF/DOCX)

Objective: Verify accurate extraction of resume content from uploaded files.

Input:

- Resume file: candidate_resume.pdf
- Resume file: candidate_resume.docx

Expected Result:

Text content is extracted correctly without missing sections or corruption.

Negative Case:

Upload a corrupted or unsupported file format.

Expected Outcome:

System handles the error gracefully and returns empty or partial text without crashing.

Sample Test Case 2 – Semantic Similarity Calculation

Objective: Validate similarity score generation between job description and resume.

Input:

- Job Description: “Looking for a Python developer with SQL and ML skills”
- Resume Text: Contains Python, SQL, Machine Learning projects

Expected Result:

System generates a cosine similarity score between 0 and 1 indicating strong relevance.

Negative Case:

Resume text unrelated to job description.

Expected Outcome:

Similarity score close to 0, indicating poor match.

Sample Test Case 3 – Skill Extraction

Objective: Ensure required skills are correctly identified from resume text.

Input:

- Resume Text: “Experienced in Python, SQL, Docker, and AWS”
- Skill List: Python, SQL, Java, ML

Expected Result:

Extracted skills = [Python, SQL]

Negative Case:

Resume does not mention any listed skills.

Expected Outcome:

Empty skill list returned.

Sample Test Case 4 – Weighted Score Calculation

Objective: Verify correct computation of final weighted score.

Input:

- Base similarity score = 0.75
- Matched skills = Python, SQL
- Skill weights = {Python: 2, SQL: 1}

Expected Result:

Final weighted score calculated correctly using 70% similarity and 30% skill weight ratio.

Sample Test Case 5 – Email Extraction

Objective: Validate candidate email extraction from resume text.

Input:

- Resume Text contains john.doe@gmail.com

Expected Result:

Extracted email = john.doe@gmail.com

Negative Case:

Resume text contains no email.

Expected Outcome:

System returns an empty string.

All unit tests passed successfully, ensuring that individual functions perform correctly in isolation.

6.2 Integration Testing and Test Cases

Integration testing was conducted to ensure smooth interaction between modules such as resume upload, text extraction, similarity computation, skill matching, score calculation, database storage, and result visualization.

Sample Integration Test Case 1 – Resume Upload → Screening → Result Display

Objective: Validate complete screening workflow.

Procedure:

1. HR logs in
2. Selects job description
3. Uploads multiple resumes
4. Runs screening

Expected Result:

Each resume is processed, scored, and displayed with match percentage and status.

Outcome: Passed successfully.

Sample Integration Test Case 2 – Screening → Database Storage

Objective: Ensure screening results are stored correctly in the database.

Procedure:

Run screening and check screening_results table.

Expected Result:

Records stored with resume name, match score, weighted score, status, skills, and timestamp.

Outcome: Passed with no data mismatch.

Sample Integration Test Case 3 – Skill Weight Input → Score Adjustment

Objective: Validate effect of skill weights on final score.

Procedure:

Enter skill weights (e.g., python:2, sql:1.5) and run screening.

Expected Result:

Candidates with weighted skills receive higher final scores.

Outcome: Passed successfully.

Sample Integration Test Case 4 – Email Notification Module

Objective: Verify email delivery to shortlisted candidates.

Procedure:

Configure SMTP → Select “Shortlisted only” → Send emails.

Expected Result:

Emails sent successfully to valid candidate email addresses.

Outcome: Passed with valid SMTP credentials.

Integration testing confirmed smooth data flow and reliable interaction between all modules.

6.3 System Testing

System testing validated the fully integrated Resume Screening System from the end-user perspective, covering functional, performance, usability, and security aspects.

Functional Testing

- Users can register, log in, and access role-based dashboards
- HR can create job postings and screen resumes
- Candidates can view their screening results
- Admin can view users, jobs, and screening history

All functional requirements were met successfully.

Performance Testing

Objective: Evaluate system responsiveness under load.

Procedure:

Upload 20 resumes simultaneously for screening.

Expected Result:

Each resume processed within 2–3 seconds without system lag.

Outcome: Passed with acceptable performance.

Security Testing

Objective: Verify role-based access control.

Procedure:

Attempt to access Admin panel as HR or Candidate.

Expected Result:

Access denied and redirected appropriately.

Outcome: Passed.

Sample System Test Case – End-to-End Workflow

Objective: Validate complete system operation.

Procedure:

1. HR creates job posting
2. Uploads resumes
3. System screens and ranks candidates
4. Shortlisted candidates stored and emailed

Expected Result:

End-to-end workflow completes without errors and accurate results are generated.

Outcome: Passed successfully.

Validation and Results

All test cases produced expected outcomes, confirming that the Resume Screening System meets its functional and non-functional requirements. The system demonstrated high accuracy in resume matching, reliable database operations, secure access control, and good usability. Testing validates that the application is ready for real-world deployment in recruitment processes.

CHAPTER 7

RESULTS AND DISCUSSION

The developed **Resume Screening System using Data Mining** produced successful outputs across multiple modules, demonstrating accurate resume analysis, efficient candidate ranking, and a user-friendly interface. The system effectively extracted resume content, analyzed skill relevance, computed similarity scores with job descriptions, and generated reliable screening results, ensuring accurate and automated candidate shortlisting.

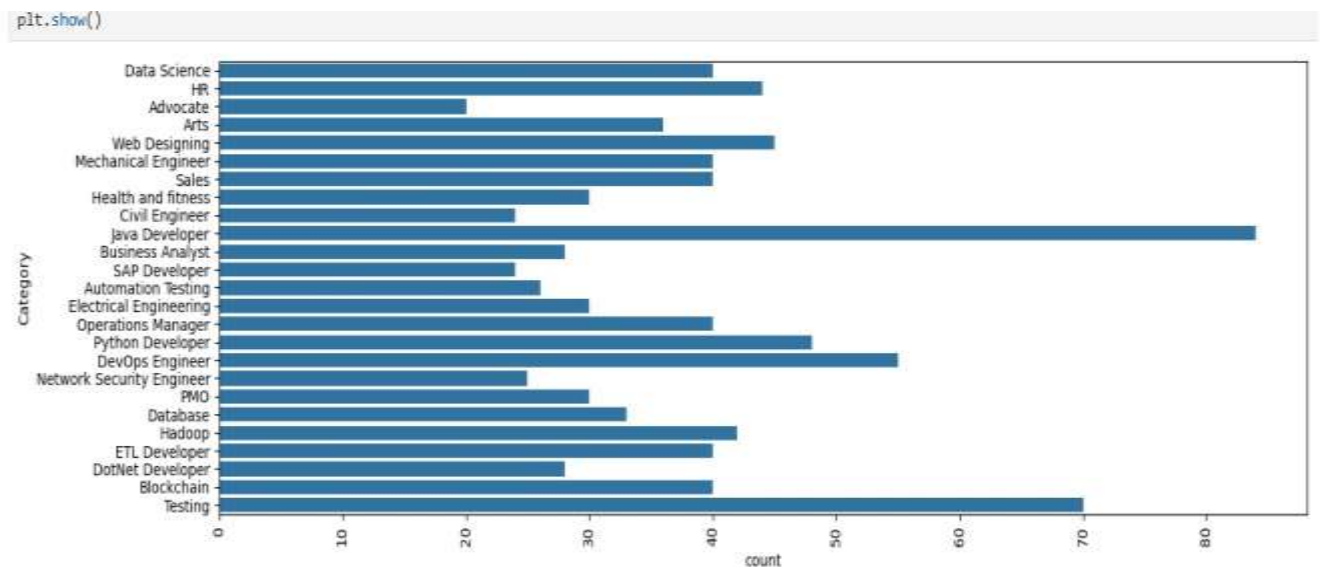


Fig 7.1 Category-wise Resume Count Bar Chart

The figure presents a horizontal bar chart showing the frequency of resumes across various job categories in the dataset. Each bar represents the number of resumes belonging to a specific domain such as Data Science, Java Developer, Automation Testing, Python Developer, DevOps, Network Security, Electrical Engineering, and several non-technical categories like HR, Sales, Arts, and Advocate. The length of each bar corresponds to the count of resumes in that category, allowing easy comparison of domain popularity. From the chart, categories such as Java Developer, Testing, and Python Developer show significantly higher counts, indicating their dominance in the dataset. This visualization helps in understanding dataset distribution, identifying high-demand job roles, and analyzing the diversity of candidate profiles available for machine learning tasks in resume screening.

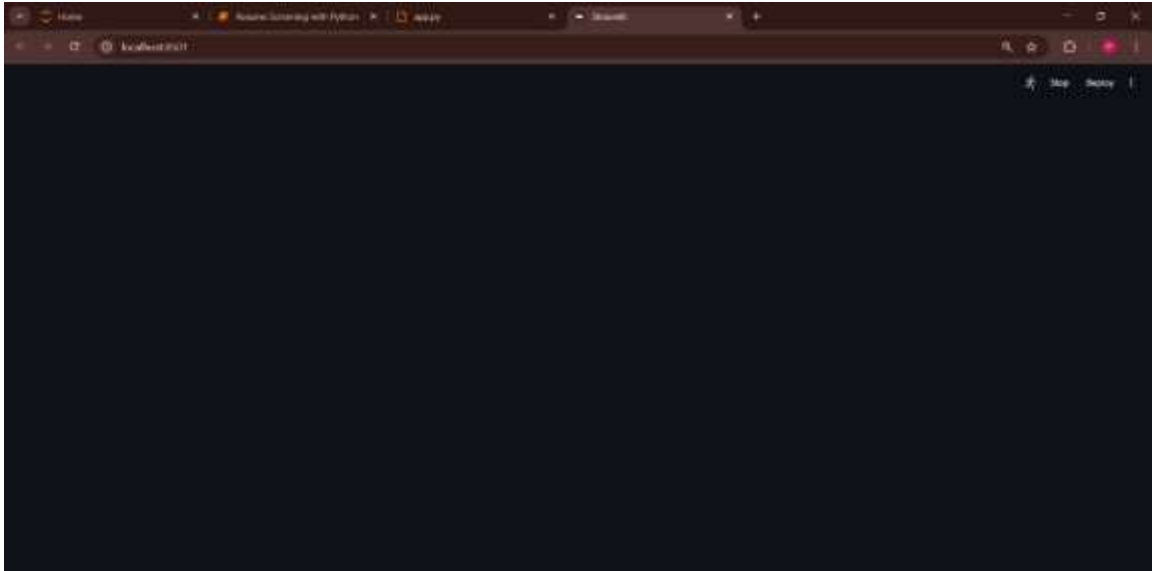


Fig 7.2 The Dashboard

The figure shows the homepage for the locally hosted Resume Screening Dashboard from localhost:8501 through Streamlit. The interface is dark-themed, as suggested that the application has been successfully initiated; this is because the user interface intends to provide the user with this theme. This is the entry point to the Resume Screening System through which users such as HR practitioners or administrators can be allowed to interact with various modules like posting a job, uploading resumes, ranking applicants, and analyzing results. The scene is blank at this moment, but it is the first layout of the dashboard in which components such as navigation menus, charts, and screening results will be rendered.

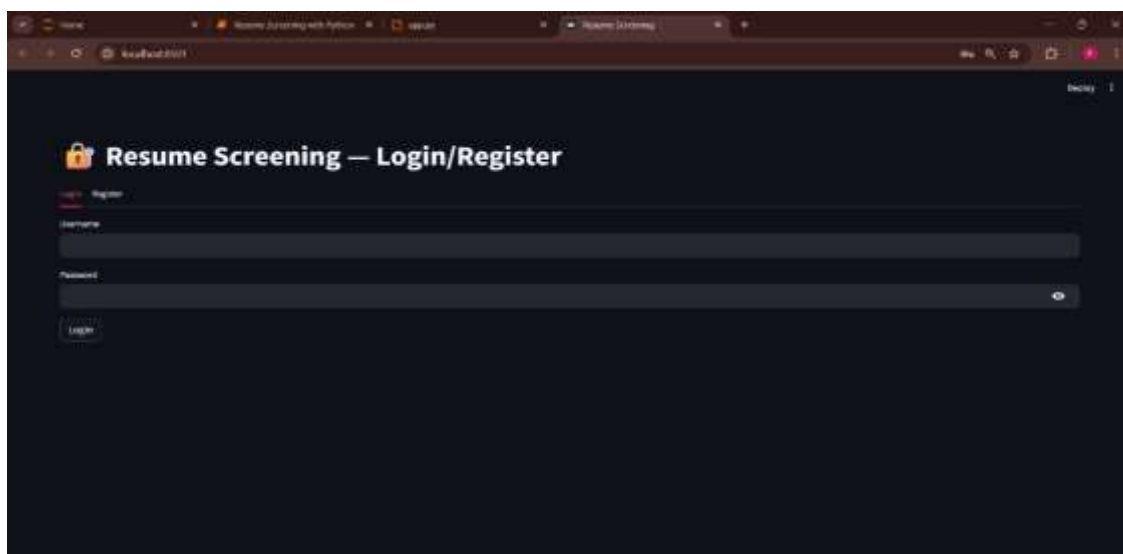


Fig 7.3 Login/Register Interface

The above figure is the Login/Register interface of Resume Screening System built using Streamlit and deployed at localhost:8501. The page acts as a secure access gateway to all its users, including HR personnel, administrators, agents, and candidates. However, the functionality it provides is broad and has two main tabs, i.e. Login and Register, used for existing users to sign in while new users create accounts. It includes input fields for Username and Password, an alternate option for revealing or hiding the password for convenience and security. A dedicated Login button initiates user verification and grants access dashboard functionalities, i.e. job posting, resume upload, candidate ranking, and analytics. This maintains the control over access while ensuring the privacy of sensitive recruitment-related.

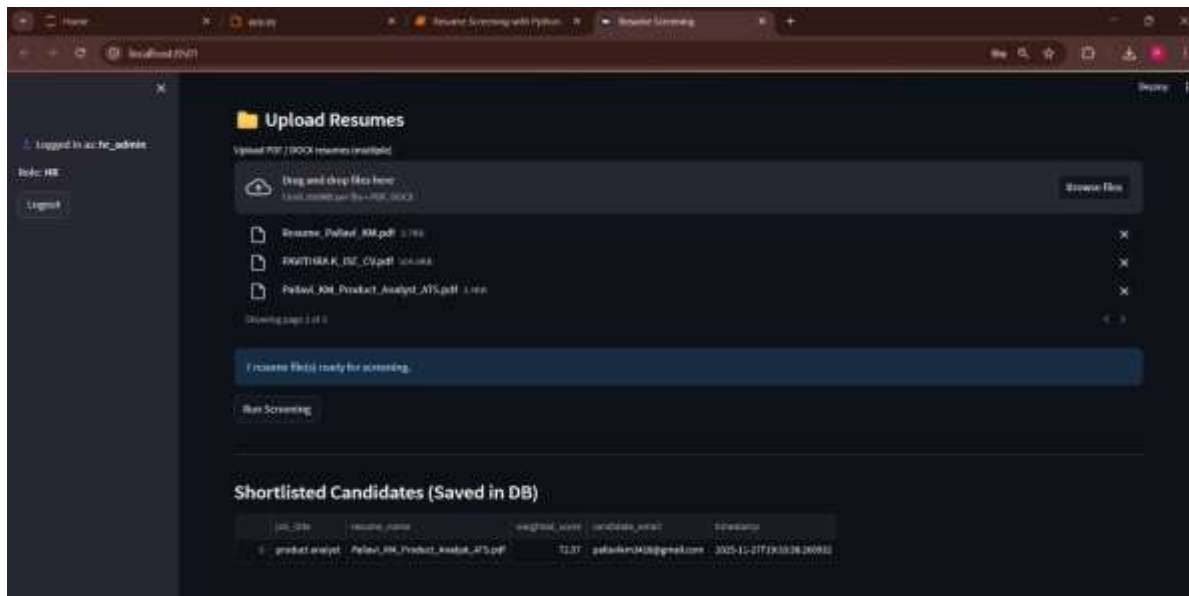


Fig 7.4 Resume Upload and Shortlisting Dashboard

This figure represents the Login and Register screens of the Resume Screening System, developed using Streamlit and hosted on localhost:8501. The page acts as the secure authentication portal for all users of the system, including the HR personnel, administrators, agents, and candidates. The interface provides two separate tabs: Login for existing users to authenticate themselves and Register for new users wishing to create an account. Input fields are provided to take in the Username and Password, with an option for toggling the visibility of the password for user convenience and security. The Login button set keeps to initiate the authentication process and subsequently allows authenticated users to access functionalities pertaining to the dashboard, such as posting of jobs, uploading of resumes, candidate ranking as well as analytics. This layer of authentication provides controlled access and protection for sensitive data related to recruitment in the system.

Below the interface for uploading, the Shortlisted Candidates (Saved in DB) table populates the essential

screening results such as Job Title, Resume File Name, Weighted Score, Candidate Email, and Timestamp. It elaborates on how the system retrieves screened outputs into an organized form of a database so that HR users can efficiently review, filter, and export the shortlisted results. The figure represents the system's crucial functionalities, which in turn include multipart file upload, automatic screening, and shortlist management through a database.

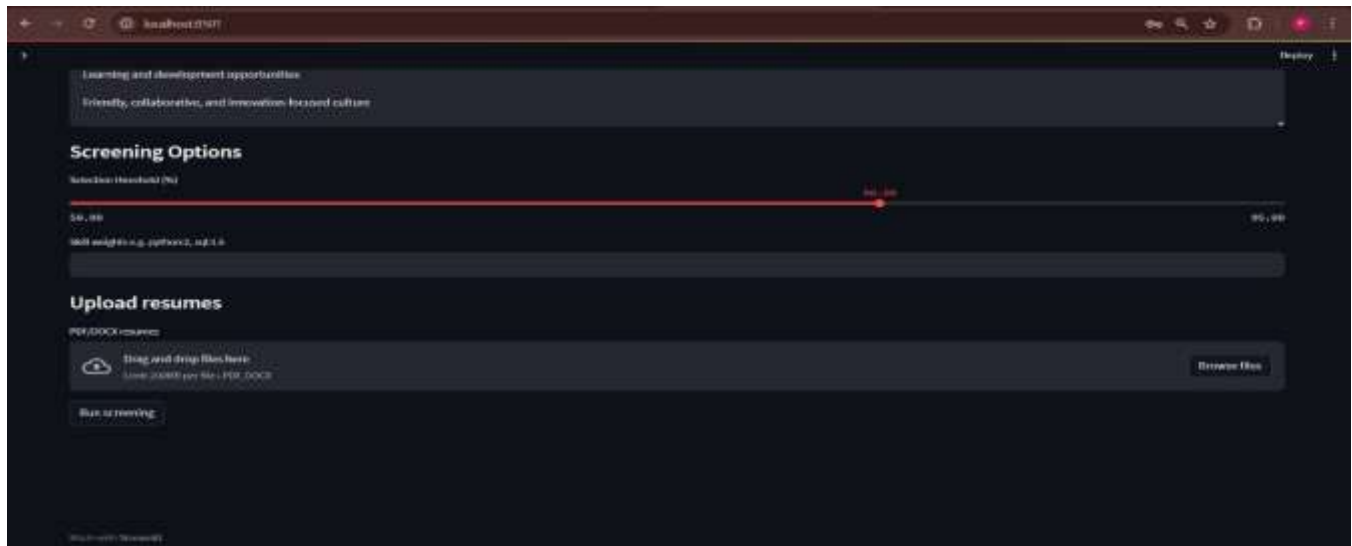


Fig 7.5 Screening Options and Resume Upload Interface

This is the Screening Options Panel in Resume Screening System, where HR users configure the important parameters influencing the automated assessment of resumes of candidates. Here, users are given the Selection Threshold Slider to set the minimum score for any candidate to become eligible for short-listing. The HR managers would be able to change this rigidity over job requirements as per the adjustment of this slider with its range from 50% to 95%. In addition to that, a text field is given for the assignment of Custom Skill Weights (e.g., python:2, sql:1.5), which would allow domain-specific prioritisation about required important technical skills. Below this screening configuration panel, of course, one is the Up Load Resumes segment, where one could easily upload PDF or DOCX file extensions through drag-and-drop or press the "Browse files" button interface. It can take bulk upload capabilities for a limit of 200 MB per file, and then the Run screening button at the bottom will be able to automatically parse, score, and rank the uploaded resumes according to the configured threshold and skill weights.

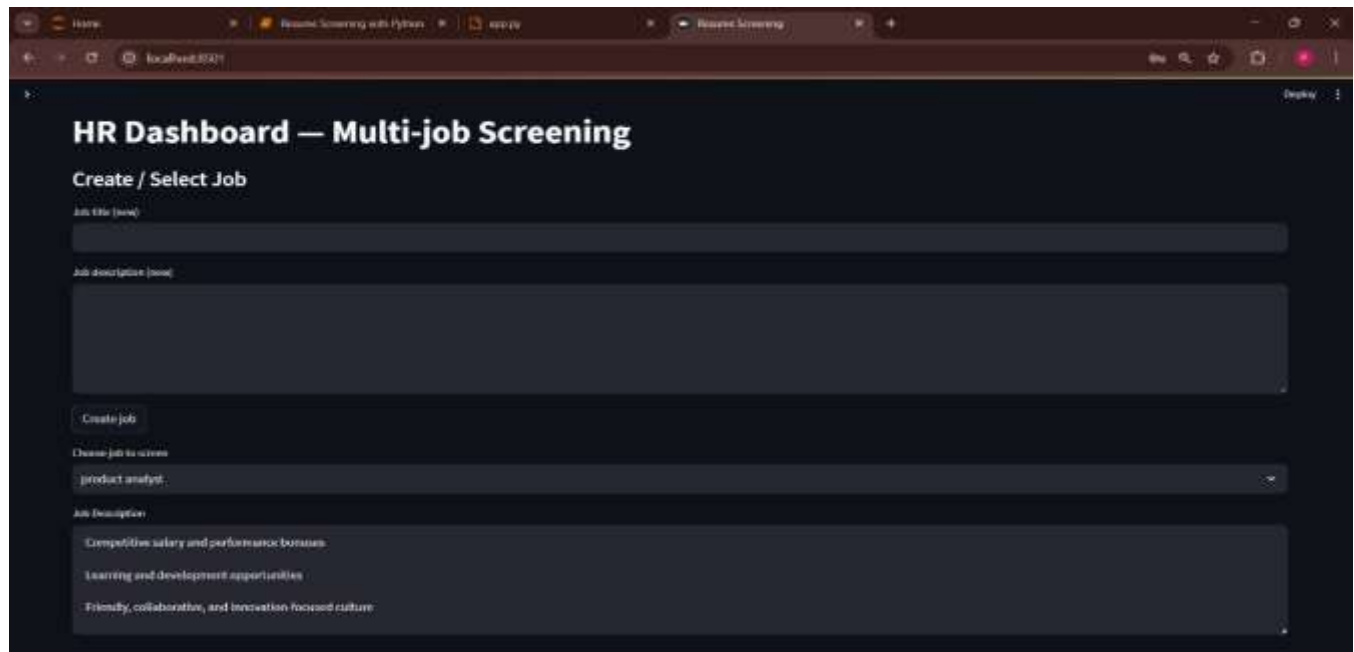


Figure 7.6 HR Dashboard: Multi-Job Screening Interface

The above figure depicts the Multi-Job Screening Dashboard of the Resume Screening System, meant for HR users to manage multiple job roles seamlessly in a single interface. On top of the dashboard is provided an option for creating new job postings by entering a job title and a detailed job description. These entry fields are meant to allow HR personnel to define customized requirements, responsibilities, and expectations for each job role. Once the relevant details are inputted, the Create Job button will send the job configuration to the system database.

Below this, the dashboard provides a drop-down listing for Choose job to screen, which would list already existing job roles that are stored in the system. Upon selecting a job(e.g., Product Analyst) the job description associated with it will come up dynamically, allowing the HR team to validate or make changes to role-specific information before commencing screening. On the job description panel, important insights will be put forth related to salary benefits, growth opportunities, and workplace culture, so as to be able to clarify this internally and for the candidates.

This interface demonstrates a system's capability to support multiple role recruitment workflows, allowing the HR teams to move seamlessly between job profiles to manage descriptions and maintain a structured hiring process across multiple positions.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 Conclusion

Project "Resume Screening Using Data Mining" demonstrates modern data-driven techniques in streamlining and automating the recruitment process. Incorporating data mining, natural language processing, and machine learning, the system can extract relevant information from resumes, use them to evaluate candidates against predefined criteria, and rank the candidates objectively and more consistently. This reduces manual effort and bias, and speeds up the initial shortlisting phase for HR teams.

The model intelligently analyzes the skills, experience, keywords, and similarity scores to match both candidates and the job. Furthermore, the interactive dashboard adds usability by allowing HR personnel to upload resumes, manage job roles, create screening thresholds, and efficiently check shortlisted candidates all in one platform. The structured database, visual analytics, and real-time processing provide accuracy and clarity in evaluating candidates.

The project as a whole showcases the practicality of data mining in recruitment-from a scalable solution to efficient, accurate, and fair candidate selection. Future improvements may include enhanced advanced semantic analysis, faster integration with applicant tracking systems, and real-time internal feedback mechanisms to further improve HR decision-making.

8.2 Future Work to Be Carried Out

While the Resume Screening System in its present design utilizing Data Mining automates the resume analysis and scoring process as well as candidate selection, it can be further developed in the future in a way to improve accuracy, scale, and usability of the system in the real world.

1. Machine Learning at a More Advanced Level

In the later version of the system, the integration of transformer models fine-tuned at the domain level and trained on recruitment datasets may help in the improvement of semantic understanding of job roles,

experience levels, and contextual skill relevance beyond the elementary consideration of similarity scoring.

2. Analysis of Experience and Education

The focus of the system is primarily on skills and semantic similarity. This can be built upon for future work by integrating experience extraction, educational qualification matching, and years-of-experience scoring methods using Named Entity Recognition (NER) techniques for a more holistic evaluation of candidates.

3. Parsing and Structuring of Resumes

The system may be able to extend the capacity for parsing resumes in a structured format by automatically identifying sections relating to skills, projects, certifications, internships, and achievements that would lead to precision in scoring and section-wise comparisons against job descriptions.

4. Reduction of Bias and Fair Screening

Future versions may include bias detection and mitigation techniques to enable a fair screening of all candidates regardless of gender, age, or background. Anonymized processing of resumes can be promoted so that ethical and unbiased hiring is assured.

5. Multi-Languages Resume Support

Currently, the majorly supported language for resume processing is English. The future versions will have the capability to process resumes in multiple languages supported by language detection and translation models for global recruitment prospective.

6. Real-Time Analytics and Dashboards

Advanced dashboards with real-time analytics, hiring trends, skill gap analysis, and performance metrics can be designed to enable HR and Admin users to support data-driven decision-making.

7. Deployment and Scalability on a Cloud

Presently the application can be deployed on cloud platforms(AWS/Azure/GCP), provided with scalable databases and storage solutions, so as to efficiently accommodate large volumes of resumes from different recruiters and concurrent users.

8. AI-Powered Recommendation System

A recommendation engine can be built to match candidates that best fit, recommend missing skills back to candidates, and suggest jobs automatically using historical screening data.

9. Enhanced Security and Authentication

The future development may encompass encrypted password storage, OAuth authentication, role-based access policies, and audit logs to enhance overall system compliance with data protection standards.

10. Interview Scheduling Integration

The system could also include automatic interview scheduling, calendar integration, and tracking of interview feedback to complete the recruitment life cycle within one single platform.

REFERENCES

- [1] Sarumathi, S., Bhavatharany E, Sanjaiy R, and Mathinesan P. “Data Driven Resume Analyzer for Student Career Optimization”. Proceedings of the 6th International Conference on Mobile Computing and Sustainable Informatics (ICMCSI-2025), IEEE, 2025, doi:10.1109/ICMCSI64620.2025.10883246.
- [2] Pant, Dipendra, Dhiraj Pokhrel, and Prakash Poudyal. “Automatic Software Engineering Position Resume Screening Using Natural Language Processing, Word Matching, Character Positioning, and Regex”. 2022 5th International Conference on Advanced Systems and Emergent Technologies (IC_ASET), IEEE, 2022, doi:10.1109/IC_ASET53395.2022.9765916.
- [3] Akram, Nida, et al. Automation of Resume Classification Using Machine Learning Algorithm. 2023 IEEE 21st Student Conference on Research and Development (SCORED), IEEE, 2023, pp. 403–407. doi:10.1109/SCORED60679.2023.10563408.
- [4] Luo, Yong, et al. Resume Net: A Learning-Based Framework for Automatic Resume Quality Assessment. 2018 IEEE International Conference on Data Mining (ICDM), IEEE, 2018, doi:10.1109/ICDM.2018.00046.
- [5] Valdez-Almada, Rogelio, et al. Natural Language Processing and Text Mining to Identify Knowledge Profiles for Software Engineering Positions: Generating Knowledge Profiles from Resumes. 2017 5th International Conference in Software Engineering Research and Innovation (CONISOFT), IEEE, 2017, doi:10.1109/CONISOFT.2017.00019.
- [6] Amin, Sujit, et al. Web Application for Screening Resume. 2019 International Conference on Nascent Technologies in Engineering (ICNTE 2019), IEEE, 2019, doi:10.1109/ICNTE44896.2019.8945947.
- [7] Nisha, B., Mano bharathi V., and Jayarajanandhini B. HR Tech Analyst: Automated Resume Parsing and Ranking System through Natural Language Processing. Proceedings of the Second International Conference on Automation, Computing and Renewable Systems (ICACRS-2023), IEEE, 2023, doi:10.1109/ICACRS58579.2023.10404426.

Appendix – A: Acronyms

Acronym	Full Form
AI	Artificial Intelligence
ATS	Applicant Tracking System
CSV	Comma-Separated Values
CV	Curriculum Vitae
DB	Database
DOC/DOCX	Microsoft Word Document
HR	Human Resources
IDE	Integrated Development Environment
JD	Job Description
JSON	JavaScript Object Notation
ML	Machine Learning
NER	Named Entity Recognition
NLP	Natural Language Processing
PDF	Portable Document Format
ROC	Receiver Operating Characteristic
SBERT	Sentence-Bidirectional Encoder Representations from Transformers
SQL	Structured Query Language
SVM	Support Vector Machine
TF-IDF	Term Frequency – Inverse Document Frequency
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator